

NexCart

E-commerce Recommendation System

Final Project Report - Local PostgreSQL + Streamlit

Field	Details
Course	Z2004 Database Management Systems
Track	Track B - AI Recommendation Engine
Database Used	PostgreSQL 18.3
Application Layer	Local Streamlit dashboard and Python recommendation API logic
Repository	https://github.com/Dexter087/Nexcart.git
Environment	Windows 11, Intel i5 CPU, 11.7 GB RAM
Team Members	Daksha Mothukuri - ZDA24B030; Manoj Phani Varma Vadapalli - ZDA24B040; V S S Preeti Ananya Yamali - ZDA24B002

Project Summary

NexCart is a functional database-backed recommendation system for a multi-seller e-commerce setting. It uses the Olist marketplace dataset, a normalized PostgreSQL schema, SQL analytics, index-based performance benchmarking, a stored procedure for product recommendation scores, and a Streamlit dashboard that produces top-N recommendations for selected customers.

Final Deliverable	Included Evidence
Database	schema.sql, ER diagram, imported Olist tables
Queries	queries.sql, performance.sql, recommendation_queries.sql
Application	Streamlit app, Python recommendation logic, FastAPI/CLI entry point
Report	This PDF report with schema, queries, performance, limitations, future work
Demo readiness	Local app commands, prepared performance evidence, demo-video placeholder

1. Abstract and Project Objective

The main goal of NexCart is to show that a recommendation system can be built on top of a relational database without hiding the important logic inside an unexplained black-box service. The final system stores e-commerce transactions in PostgreSQL and uses those transactions to produce top-N product recommendations for a given customer. The project demonstrates both database engineering and application-level integration: schema design, data loading, query coverage, indexing, a stored procedure, and a Python user interface.

The recommendation module uses user-based collaborative filtering. A target customer is compared with other customers based on overlapping product and category purchases. Candidate products bought by similar customers are then ranked with SQL window functions. This keeps the recommendation logic auditable because the similarity, candidate generation, ranking, and fallback strategy can all be inspected as SQL.

2. Requirement Mapping

The following matrix shows how the final implementation satisfies Track B requirements and the final submission checklist.

Requirement	NexCart Implementation	Evidence File/Feature
Normalized database with 5+ tables	Seven core tables: customers, sellers, products, orders, order_items, order_payments, order_reviews.	schema/schema.sql; ER diagram
2000+ transactional rows	112,650 order_items and 99,441 orders were loaded into PostgreSQL.	performance_output.txt row-count section
Collaborative filtering with SQL window functions	CTE pipeline uses SUM OVER PARTITION BY, RANK, and ROW_NUMBER.	queries/recommendation_queries.sql; app/recommender.py
Python API returns top-N recommendations	Recommendation logic is available through app/main.py and used by Streamlit.	app/main.py; app/streamlit_app.py
Benchmark with and without indexes	performance.sql drops selected indexes, measures queries, creates indexes, and measures again.	queries/performance.sql
Stored procedure/trigger	refresh_product_recommendation_scores() rebuilds product-level recommendation scores.	queries/performance.sql

This mapping is important for the final viva because it connects every technical claim to a visible artifact in the repository. The project is therefore not only a written description; it is a reproducible system with database scripts and executable application code.

3. Dataset Description and Loading Outcome

NexCart uses the Olist Brazilian E-Commerce Public Dataset as its primary data source. The dataset is suitable because it contains realistic marketplace relationships: customers place orders, orders contain items, items are sold by sellers, products belong to categories, and orders have payments and reviews. This makes it a strong relational base for both SQL analysis and recommendation logic.

The final implementation uses seven CSV files. The geolocation file was not used because the recommendation workflow did not require latitude-longitude records. The category translation file was also not used in the working schema because product_category_name is stored directly in products, which avoided import problems caused by category mismatches.

CSV/Table	Rows Loaded	Purpose in NexCart
olist_customers_dataset.csv / customers	99,441	Customer IDs, unique-customer grouping, city, and state.
olist_sellers_dataset.csv / sellers	3,095	Seller IDs and seller location details.
olist_products_dataset.csv / products	32,951	Product IDs, categories, and physical product attributes.
olist_orders_dataset.csv / orders	99,441	Order headers, customer links, order status, and timestamps.
olist_order_items_dataset.csv / order_items	112,650	Core transaction table linking orders to

		products, sellers, price, and freight.
olist_order_payments_dataset.csv / order_payments	103,886	Payment type, installments, and payment value.
olist_order_reviews_dataset.csv / order_reviews	99,224	Review scores and optional review text fields.

The row-count check confirms that the dataset is far above the minimum size needed for Track B. The most important table for recommendation is order_items because each row is a purchase interaction between a customer order and a product.

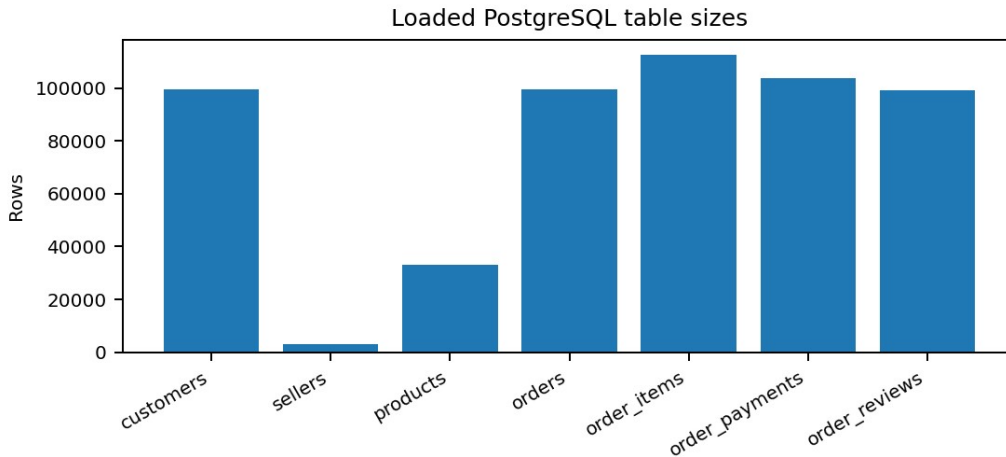


Figure 1. Table sizes loaded into PostgreSQL.

4. Relational Schema and Normalization

The schema separates master entities from transactional relationships. Customers, sellers, and products are stored once in their own tables. Orders store the order header and link to customers. Order_items represents the many-to-many relationship between orders and products, also storing transaction-specific facts such as price, seller, shipping limit date, and freight value. Payments and reviews are separated from the order header because they describe different aspects of an order.

This design supports 3NF because each non-key attribute depends on the key of its own table, not on unrelated attributes from another table. For example, seller_city depends on seller_id and remains in sellers, while price depends on the specific sold order item and remains in order_items.

Table	Primary Key	Functional Dependency / Design Note
customers	customer_id	customer_id -> customer_unique_id, customer_zip_code_prefix, customer_city, customer_state
sellers	seller_id	seller_id -> seller_zip_code_prefix, seller_city, seller_state
products	product_id	product_id -> category, name length, description length, photos, weight, dimensions
orders	order_id	order_id -> customer_id, status, purchase/approval/delivery timestamps
order_items	order_id + order_item_id	The key identifies one line item; price and freight belong to that item, not only to product_id.
order_payments	order_id + payment_sequential	The key supports multiple payments for one order.
order_reviews	review_id + order_id	The composite key handles duplicate review_id cases in the source data.

4.1 Constraint Strategy

Primary keys protect entity identity. Foreign keys preserve relationships between orders, customers, products, sellers, payments, and reviews. CHECK constraints prevent invalid numeric values such as

negative price, freight, dimensions, and review scores outside the 1-5 range. These constraints make the imported data usable for analysis and reduce the risk of inconsistent recommendations.

5. Final ER Diagram and Relationship Explanation

The ER diagram below represents the final implemented schema. The central transactional chain is customers to orders to order_items. Order_items connects products and sellers, making it the strongest table for recommendation and seller analytics. Payments and reviews connect to orders, while product_recommendation_scores connects to products as a derived analytical table.

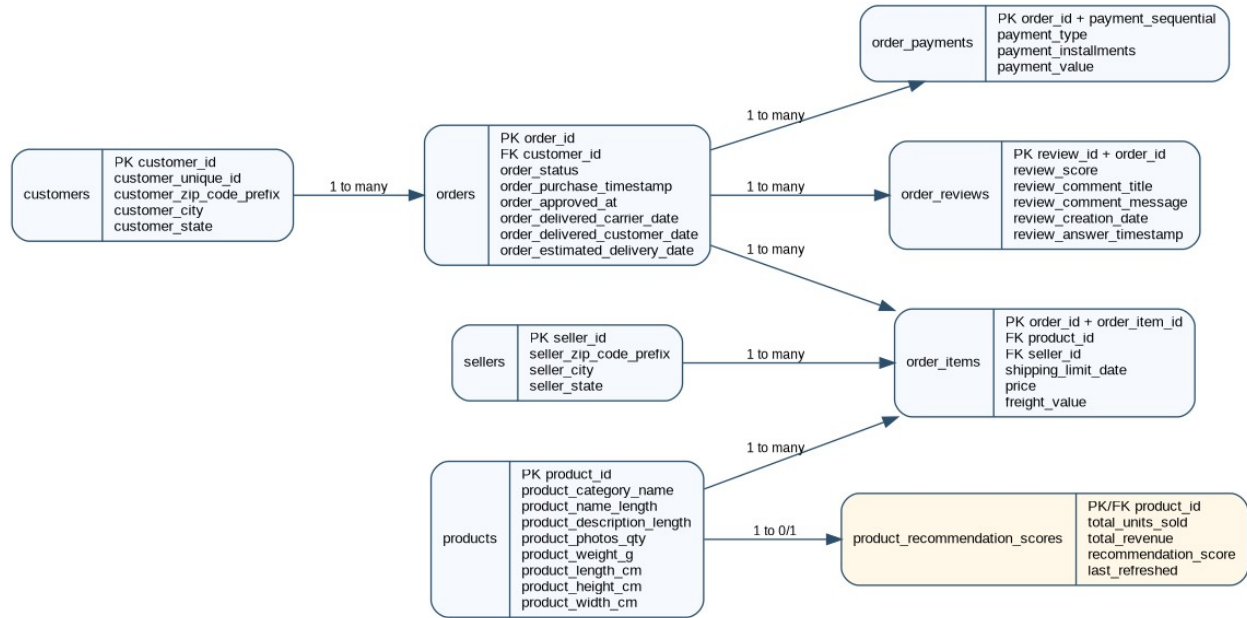


Figure 2. Final NexCart PostgreSQL schema diagram.

Relationship	Cardinality	Reason
customers -> orders	1 to many	One customer can place many orders; each order belongs to one customer_id.
orders -> order_items	1 to many	One order can contain several purchased products.
products -> order_items	1 to many	A product can appear in many order items across the marketplace.
sellers -> order_items	1 to many	A seller can sell many order items.
orders -> payments/reviews	1 to many	An order can have payment records and review records in the source data.
products -> product_recommendation_scores	1 to 0/1	A product may have one derived score row after procedure refresh.

6. Data Dictionary and Cleaning Decisions

The data dictionary below summarizes the major columns used in the project. Keeping these meanings documented is necessary for reproducibility and helps explain query outputs during the final demo.

Table	Important Columns	Meaning
customers	customer_id, customer_unique_id, city, state	customer_id identifies order-level customer rows; customer_unique_id groups repeat customers.
sellers	seller_id, seller_city, seller_state	Seller identity and location fields used in seller analytics.
products	product_id, product_category_name, product_weight_g, product dimensions	Product identity, category, and physical product attributes.
orders	order_id, customer_id, order_status, purchase/delivery timestamps	Order lifecycle and customer relationship.
order_items	order_id, order_item_id, product_id, seller_id, price, freight_value	Purchase interactions used for recommendation, revenue, and seller analysis.
order_payments	payment_type, payment_installments,	Payment behavior and payment-value analysis.

	payment_value	
order_reviews	review_score, comment fields, creation/answer timestamps	Customer satisfaction and review analysis.

6.1 Import Notes

During loading, order_reviews required an encoding-safe import because review text can include special characters. After resolving the encoding issue, the final count for order_reviews was 99,224 rows. This confirms that the review table was loaded successfully rather than dropped or reduced to avoid errors.

The final database name used for local reproducibility is nexcart_olist. Dataset files are stored in dataset_olist in the user environment, and the loading script is written to detect either dataset_olist or data so the repository remains flexible.

```
psql -U postgres -h localhost -p 5432 -d nexcart_olist -c "\copy order_reviews FROM 'dataset_olist/olist_order_reviews_dataset.csv' WITH (FORMAT csv, HEADER true, ENCODING 'LATIN1')"
```

7. SQL Query Design

The queries.sql file demonstrates the required variety of SQL operations and gives meaningful e-commerce outputs. The file is structured from simpler aggregation and joins toward subqueries, CTEs, and window functions.

Query	Type	Purpose
Q1	Aggregation	Counts total orders by customer state.
Q2	Aggregation	Computes average payment value by payment type.
Q3	Join	Combines orders with customer city and state.
Q4	Join	Connects order_items with sellers.
Q5	Subquery	Finds customers above the average order count.
Q6	Subquery	Finds products that never appear in order_items.
Q7	CTE	Calculates delivery delay for delivered orders.
Q8	CTE	Counts sold items by product category.
Q9	Window function	Ranks customers by total spending.
Q10	Window function	Ranks products within each category by average selling price.

These queries are useful in a real e-commerce context. State-level order counts identify geographic demand. Payment summaries reveal payment behavior. Delivery delay analysis supports operational monitoring. Customer spending rank identifies high-value customers, while category-specific product ranking helps avoid comparing products across unrelated categories.

```
-- Run all analytical queries
psql -U postgres -h localhost -p 5432 -d nexcart_olist -f queries/queries.sql
```

7.1 Why Window Functions Matter

Window functions are important because they calculate rankings while preserving row-level results. Instead of collapsing every row into a single aggregate, they allow the system to rank customers and products inside meaningful partitions.

8. Recommendation Engine Design

The recommendation engine uses SQL-based user collaborative filtering. It starts with a selected customer_id, maps it to customer_unique_id, finds purchased products and categories, and then searches for other customers with overlapping behavior. Product overlap is given a higher weight than

category overlap because buying the exact same product is stronger evidence of similarity than buying within the same category.

Pipeline Stage	Main SQL Component	Explanation
Target customer	target_customer	Locates the selected customer and repeat-customer identifier.
Target orders	target_orders	Collects valid orders for the same customer_unique_id.
Target purchases	target_purchases	Finds already purchased products and categories.
All interactions	all_interactions	Builds the customer-product-category interaction table.
Similarity	overlap_events, overlap_scores	Scores other customers using product and category overlap.
Candidate generation	candidate_products	Removes products already purchased by the target customer.
Ranking	ranked_products	Ranks candidates and controls category repetition.

The ranking uses RANK to create the main recommendation order and ROW_NUMBER partitioned by product_category_name so the output is not dominated by one category. If collaborative filtering returns no rows because a customer is too sparse, the application uses a popularity fallback ranked by total units sold and revenue.

```
SUM(overlap_weight) OVER (PARTITION BY similar_customer_unique_id) AS similarity_score
RANK() OVER (ORDER BY collaborative_score DESC, similar_customer_count DESC) AS recommendation_rank
ROW_NUMBER() OVER (PARTITION BY product_category_name ORDER BY collaborative_score DESC) AS category_row_number
```

8.1 Recommendation Output

The app returns product_id, product_category_name, average price, similar-customer count, similar-purchase count, recommendation score, and rank. These fields make the recommendation explainable: a user can see not only which product is recommended, but also why it was ranked highly.

9. Streamlit Application and Python API Design

The final application is a local Streamlit dashboard connected to the local PostgreSQL database. Streamlit was chosen because it gives a clear demo interface: the user can switch between database summary, recommendations, SQL analytics, performance evidence, and stored-procedure refresh without relying on command-line output alone.

File	Responsibility	Demo Evidence
app/db.py	Reads .env values and opens PostgreSQL connections.	Shows no hard-coded password.
app/recommender.py	Runs collaborative filtering SQL and fallback ranking.	Recommendation page calls this logic.
app/analytics.py	Provides row counts, chart data, payment and review summaries.	Database Summary and SQL Analytics pages.
app/streamlit_app.py	Defines visual pages and user controls.	Main demo application at localhost:8501.
app/main.py	Provides API/CLI-compatible top-N recommendation interface.	Supports the Python API requirement.

9.1 App Pages

Page	What It Shows	Why It Is Useful
Home	Project overview, connection status, and workflow summary.	Introduces the system.
Database Summary	Loaded rows and charts from PostgreSQL.	Proves the database is populated.
Recommendation Engine	Customer selector, top-N setting, recommendation table.	Shows the core Track B feature.
SQL Analytics	Orders by state, category sales, payments, and review summaries.	Shows meaningful SQL outputs.
Performance Evidence	Before/after timings and index interpretation.	Shows performance benchmark.
Stored Procedure Demo	Refresh button and product score table.	Shows database-side routine.

```
streamlit run app/streamlit_app.py
```

```
# Open: http://localhost:8501
```

10. Performance Benchmarking Methodology

The performance benchmark uses the same dataset and the same SQL text before and after indexing. The script first drops only the project-specific indexes so that the before-index stage is real. It then executes EXPLAIN ANALYZE for selected queries, creates the indexes, runs ANALYZE, and repeats the queries. This method is simple, reproducible, and easy to explain in the demo.

Query	Why It Was Selected	Expected Behaviour
Customer order history	Common app operation when retrieving a user purchase record.	Should improve strongly because customer_id and order_id lookups are selective.
Top-selling products	Important analytical query and useful for fallback recommendations.	May improve moderately, but must still aggregate many rows.
Seller revenue ranking	Marketplace analytics query with seller-level grouping.	May not improve because it needs most order_items rows.

10.1 Index Design

The index choices target foreign-key columns and date fields that appear repeatedly in joins, filters, and order-by clauses. They are B-tree indexes, which are appropriate for equality joins and timestamp ordering in PostgreSQL.

Index	Column Targeted	Justification
idx_orders_customer_id	orders(customer_id)	Speeds up customer-specific order history and recommendation target lookup.
idx_order_items_order_id	order_items(order_id)	Speeds up joining order headers to their items.
idx_order_items_product_id	order_items(product_id)	Supports product sales aggregation and product-based similarity.
idx_order_items_seller_id	order_items(seller_id)	Supports seller revenue analysis.
idx_orders_purchase_timestamp	orders(order_purchase_timestamp)	Supports date-based sorting and filtering.

```
psql -U postgres -h localhost -p 5432 -d nexcart_olist -f queries/performance.sql > performance_output.txt 2>&1
```

11. Performance Results and Interpretation

The benchmark produced three useful outcomes. One query improved dramatically, one improved moderately, and one became slower. This is a stronger performance discussion than simply claiming that every index helps, because it shows that index usefulness depends on selectivity and query shape.

Benchmark Query	Before	After	Outcome	Interpretation
Customer order history	6.521 ms	0.162 ms	97.5% faster	The plan changed from scanning orders to indexed lookup using customer_id and order_id indexes.
Top-selling products	477.947 ms	305.587 ms	36.1% faster	The product_id index helped, but the query still processed all 112,650 order items.
Seller revenue ranking	69.428 ms	99.546 ms	43.4% slower	The optimizer still used hash join and sequential scan because the aggregate touches most rows.

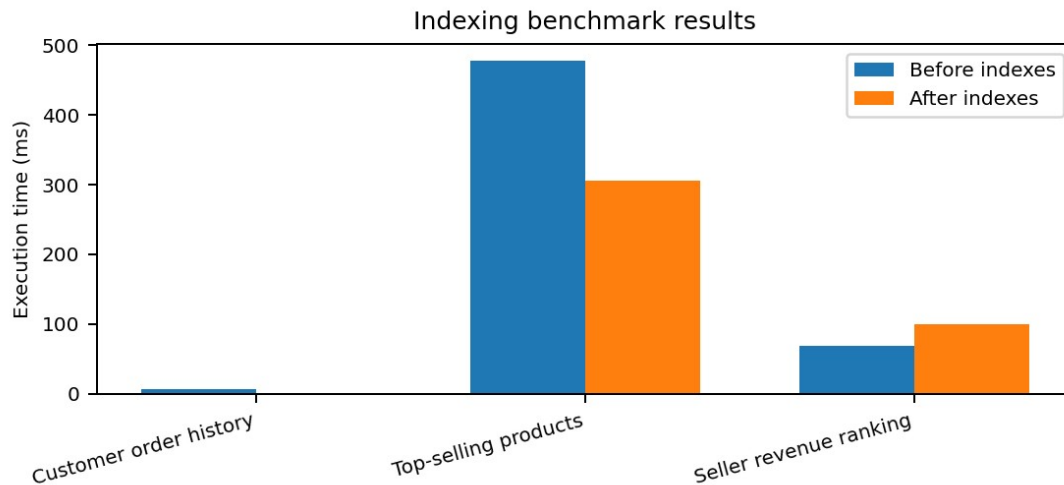


Figure 3. Measured before/after execution times from EXPLAIN ANALYZE.

11.1 Query Plan Interpretation

For the customer order history query, the before-index plan scanned orders and removed 99,440 rows by filter. After indexing, PostgreSQL used `idx_orders_customer_id` and `idx_order_items_order_id`, reducing execution time to 0.162 ms. This is the cleanest example of a useful index.

For global product and seller rankings, the database still needs to read and aggregate most rows. In such cases, sequential scans and hash aggregation can be rational choices. The seller query becoming slower after indexing does not mean the script failed; it shows that not every index improves every workload.

12. Stored Procedure and Recommendation Score Table

NexCart includes a database-side routine named `refresh_product_recommendation_scores()`. The procedure truncates and rebuilds `product_recommendation_scores` using sales information from `order_items`. The refreshed table contains `product_id`, `total_units_sold`, `total_revenue`, `recommendation_score`, and `last_refreshed`.

Object	Role	Observed Result
<code>product_recommendation_scores</code>	Stores precomputed product-level recommendation scores.	32,951 product score rows.
<code>refresh_product_recommendation_scores()</code>	Updates recommendation scores from current sales data.	CALL executed successfully.
Stored-score query	Ranks products by <code>recommendation_score</code> .	Top 10 query executed in 42.359 ms.

13. Reproducibility and Security

The repository is designed so another evaluator can reproduce the database and run the app locally. The `.env` file is excluded from version control, while `.env.example` documents the required variables. This avoids committing PostgreSQL passwords.

Step	Command/File
Create Python environment	<code>scripts\setup_env.cmd</code>
Load PostgreSQL database	<code>scripts\load_database.cmd</code>
Run Streamlit dashboard	<code>scripts\run_app.cmd</code>
Run API/CLI path	<code>scripts\run_api.cmd</code> or <code>python -m app.main</code>
Save performance evidence	<code>queries\performance.sql > performance_output.txt</code>

```
set PGPASSWORD=YOUR_POSTGRES_PASSWORD
scripts\setup_env.cmd
```



```
scripts/load_database.cmd
scripts/run_app.cmd
```

Because the app runs locally against PostgreSQL, the demo does not depend on cloud database availability. This reduces the risk of network or hosting failures during the final screen recording.

14. Demo Plan and Fallback Strategy

The five-minute demo should show setup, core features, and a DB/performance feature. A good order is: repository structure, local database row counts, Streamlit dashboard, recommendations for three customers, SQL analytics page, performance page, stored-procedure refresh, and final report overview.

Demo Segment	What to Show	Fallback
Repository and setup	README, scripts, schema, queries, app folder.	Use recorded commands if reload is slow.
Database loaded	Row counts for all seven tables.	Use performance_output.txt as evidence.
Recommendations	Generate top-N recommendations for three customers.	Use sample IDs shown by app.
Performance/DB feature	Before/after timings and stored procedure refresh.	Show performance page and SQL output.

15. Limitations

The Olist dataset is historical and does not include live browsing events, search queries, impressions, cart additions, or real-time inventory. Therefore, NexCart recommends based on purchase history rather than current user intent. Some customers are sparse, so the app includes a popularity fallback when collaborative overlap is too limited.

16. Future Work

Future work can add browsing-event tables, inventory and warehouse extensions, category translation, product text embeddings, user login flow, and hybrid ranking that combines purchase overlap, review score, delivery reliability, and price. The stored procedure could also be extended into an incremental trigger that updates product scores whenever new order_items rows are inserted.

17. AI Usage Disclosure and Conclusion

AI tools were used for brainstorming, debugging SQL and Python structure, drafting documentation, and improving report wording. The generated material was reviewed and adapted to the final NexCart schema, local PostgreSQL outputs, and Streamlit implementation before inclusion.

NexCart satisfies the final Track B direction by combining a normalized PostgreSQL database, meaningful transactional data, labelled query files, SQL-based collaborative filtering, performance evidence, a stored procedure, and a working local Python application. The final submission is therefore a demonstrated database system rather than only a written description.

Appendix. Core Run Commands

```
cd /d C:\Users\Daksh\Downloads\Nexcart
set PGPASSWORD=YOUR_POSTGRES_PASSWORD
scripts/load_database.cmd
scripts/run_app.cmd
# Browser: http://localhost:8501
```

Appendix B. Extra Explanation for Viva Questions

Why PostgreSQL? PostgreSQL was selected because the project is relational at its core and requires DDL, constraints, joins, indexes, EXPLAIN ANALYZE, and stored procedures. Keeping the

recommendation logic inside SQL makes the database contribution visible and easier to defend during Q&A.

Why collaborative filtering? Collaborative filtering is suitable because the Olist dataset is transaction-heavy. It does not require rich product text or images. The user-product interactions in `order_items` already provide the behavioral signal needed to find similar customers and candidate products.

Why a fallback recommender? Sparse customers may have very few purchases. In that case, overlap-based collaborative filtering can return no candidates. The popularity fallback prevents an empty demo output and represents a common production pattern where cold-start users receive broadly popular products.

Why did some indexes not improve all queries? B-tree indexes are strongest for selective lookup. Queries that aggregate almost the entire table may still prefer sequential scans, hash joins, and hash aggregation. This is why the customer-order query improved dramatically, while the seller aggregation did not.

Appendix C. Expected Demo Test Cases

Test Case	Input / Action	Expected Result
Connection test	Open Streamlit Home page.	The app shows PostgreSQL connection success if <code>.env</code> is correct and the server is running.
Database summary	Open Database Summary page.	Seven table row counts appear, matching the loaded Olist dataset.
Recommendation output	Select a sample customer and <code>top_n=5</code> .	The app displays product recommendations with category, score, and rank.
Sparse customer fallback	Choose a customer with little purchase overlap.	The app uses the popularity fallback instead of failing.
Stored procedure	Click refresh recommendation scores.	The <code>product_recommendation_scores</code> table is refreshed and top scores are displayed.

Appendix D. Database Loading and Validation Detail

The database loading process is a critical part of reproducibility because a recommendation app is only useful if the underlying relational data is loaded consistently. The final command-prompt workflow creates a fresh `nexcart_olist` database, runs `schema/schema.sql`, imports the seven CSV files in dependency order, verifies row counts, and then runs the query and performance scripts. The import order matters because foreign-key relationships require parent tables before child tables.

The parent tables `customers`, `sellers`, and `products` are loaded first. `Orders` are loaded after `customers` because `orders.customer_id` references `customers.customer_id`. `Order_items` is loaded after `orders`, `products`, and `sellers` because it references all three. `Payments` and `reviews` are loaded after `orders` because each payment and review belongs to an order. This dependency-aware import order prevents foreign-key failures and also makes the loading script understandable to a reviewer.

Load Order	Table	Reason for Position
1	<code>customers</code>	Parent table for <code>orders</code> .
2	<code>sellers</code>	Parent table for <code>order_items</code> .
3	<code>products</code>	Parent table for <code>order_items</code> and <code>recommendation_scores</code> .
4	<code>orders</code>	Depends on <code>customers</code> and acts as parent for <code>items</code> , <code>payments</code> , and <code>reviews</code> .
5	<code>order_items</code>	Depends on <code>orders</code> , <code>products</code> , and <code>sellers</code> .
6	<code>order_payments</code>	Depends on <code>orders</code> .
7	<code>order_reviews</code>	Depends on <code>orders</code> and contains review text that may need encoding handling.

Validation is performed with a single UNION ALL row-count query. This is better than checking each table manually because it creates one compact output that can be saved in the report evidence. After

loading, the row counts also become a quick sanity check before running the app: if any table is zero or missing, the recommendation page will not behave correctly.

Appendix E. Testing and Validation Approach

NexCart was checked through database, query, and application-level tests. At the database level, the schema was run from scratch and the CSV files were loaded into PostgreSQL. At the query level, queries.sql, performance.sql, and recommendation_queries.sql were executed against the loaded database. At the application level, the Streamlit app connects to PostgreSQL, retrieves sample customers, displays table counts, runs recommendations, and calls the stored procedure.

Validation Area	Test Performed	Expected Evidence
Schema	Run schema.sql on a fresh database.	All tables and constraints are created without manual edits.
Import	Run load_database.cmd and row-count check.	Seven tables return nonzero row counts matching the Olist data.
SQL analytics	Run queries.sql.	Aggregation, join, subquery, CTE, and window-function outputs appear.
Performance	Run performance.sql and save output.	Before/after EXPLAIN ANALYZE plans and timings are recorded.
Recommendation SQL	Run recommendation_queries.sql.	Top-N candidate products are returned for a valid sample customer.
Application	Run Streamlit and test pages.	Dashboard loads, recommendations display, stored procedure refresh works.

The most important application test is to generate recommendations for at least three customers before recording the final demo. This confirms that the dashboard is not only connected to PostgreSQL but is actually executing the recommendation query and returning useful product rows. If a sparse customer returns fallback recommendations, that should be explained as the cold-start handling strategy rather than treated as an error.

Appendix F. Viva and Q&A Readiness

Several likely viva questions can be answered directly from the implementation. If asked why Streamlit was used, the answer is that Streamlit provides a clear local interface for a database-backed app and makes the demo easier to follow than command-line output alone. If asked why PostgreSQL was used, the answer is that the project requirements emphasize relational schema design, SQL, indexing, EXPLAIN ANALYZE, and stored procedures.

If asked whether this is really collaborative filtering, the answer is yes: the recommender uses user-purchase interactions, compares customers by overlapping purchases and categories, and recommends products bought by similar customers but not already bought by the target customer. The model is implemented with SQL CTEs and window functions instead of a separate ML library so the logic remains transparent.

If asked why the seller ranking got slower after indexes, the answer is that index benefit depends on selectivity. The seller ranking aggregates almost the whole order_items table, so a sequential scan and hash aggregation can be more efficient than forcing index-heavy access. This result is useful because it shows the team interpreted the plan instead of assuming indexes always improve performance.

Question	Prepared Answer
Why not use a black-box recommender library?	The rubric emphasizes relational database work, so SQL-based recommendation logic makes the database contribution visible.
What is the AI component?	The AI/recommendation component is collaborative filtering over user-product interactions, ranked with SQL window functions.
Why use customer_unique_id?	It groups repeat purchases by the same real customer across different order-level customer_id values.
What is the fallback plan?	Sparse users receive popularity-based recommendations so the app still produces meaningful output.

Appendix G. Final Submission Checklist Traceability

This appendix summarizes how the final repository should be checked immediately before submission. It is included to make the report useful during the last review and to reduce the chance of missing a required artifact.

Checklist Area	Repository Evidence	Final Check
schema/	schema.sql and er_diagram.png	schema.sql runs from scratch and diagram is readable.
dataset_olist/ or data/	Seven Olist CSV files and README_DATA.md	CSV files are not accidentally deleted during package copy.
queries/	queries.sql, performance.sql, recommendation_queries.sql	Each file is labelled and runs against nexcart_olist.
app/	streamlit_app.py, recommender.py, analytics.py, db.py, main.py	Streamlit app starts and recommendations return rows.
report/	NexCart_Final_Report.pdf	Report is under 15 pages and includes schema, queries, performance, limitations, and future work.
demo/	demo_video_link.txt or video file	Demo link/video is added after recording.
README.md	Setup, database loading, app run commands, AI usage disclosure	Commands are copy-pasteable for Command Prompt.
.env handling	.env.example and .gitignore	No database password is committed.

Before submission, the team should run git status and confirm that no local .env file, database dump containing passwords, or temporary performance scratch files are accidentally staged. The final repository should show real project structure and not just a single report upload. The app should be tested once from a fresh Command Prompt session because that is the closest match to the evaluator reproducing the work.

Appendix H. Short Technical Glossary

Term	Meaning in NexCart
3NF	A normalization target where non-key fields describe the key and avoid unnecessary transitive dependency.
CTE	A named temporary query block used to break recommendation and analytics logic into readable steps.
Window function	A SQL function that computes ranks or totals over related rows without collapsing the result set.
Collaborative filtering	Recommendation approach that suggests products from customers with similar purchase behavior.
Cold start	A sparse-user situation where there is not enough purchase overlap for personalized recommendations.
EXPLAIN ANALYZE	PostgreSQL command that runs a query and reports the actual execution plan and timing.
Stored procedure	Database routine that performs a reusable operation, such as refreshing recommendation scores.

These terms are included because they are likely to appear in Q&A. Being able to connect each term to a file and a demo action will make the final presentation more confident and easier to defend.